

# Condition Handling

[1. Condition Handling – Kompakte System-Referenz für KI-Modelle](#)

[2. Grundprinzip](#)

[3. Placeholder Resolution](#)

[4. Equality](#)

[5. Inequality](#)

[6. String-Vergleiche](#)

[7. Numerisch aussehende Werte](#)

[8. CONTAINS](#)

[9. AND](#)

[10. OR](#)

[11. NOT](#)

[12. Klammern](#)

[13. Operator-Priorität](#)

[14. && und ||](#)

[15. NULL](#)

[16. IS\\_EMPTY und IS\\_NOT\\_EMPTY](#)

[17. IS\\_NULL und IS\\_NOT\\_NULL](#)

[18. Case Sensitivity](#)

[19. Nicht unterstützte Operatoren](#)

[20. Keine Arithmetik](#)

[21. Keine Standalone-Boolean-Expressions](#)

[22. Viele praktische Beispiele](#)

[Status ist READY](#)

[Status ist nicht READY](#)

[Status ist READY oder DONE](#)

[Status ist weder FAILED noch ERROR](#)

[Zwei Werte müssen vorhanden sein](#)

[Text enthält SUCCESS](#)

[Text enthält keinen ERROR](#)

[Ergebnis ist leer](#)

[Ergebnis ist nicht leer](#)

[Count ist 0](#)

[Count ist nicht 0](#)

[Feature ist aktiv und Status READY](#)

[Einer von zwei Prozessen läuft](#)

[Status ist PENDING oder PROCESSING](#)

[Antwort vorhanden und enthält READY](#)

[Komplexere Kombination](#)

[23. Verhalten bei normalen Steps](#)

[24. Verwendung durch Control-Flow-Engines](#)

[25. Bekannte Parser-Einschränkung](#)

[26. Best Practices](#)

[27. Troubleshooting](#)

[28. Kurzreferenz für KI / RAG](#)

AGATE unterstützt über das Feld `condition` die bedingte Ausführung von Testschritten.

`condition` ist **keine Engine-spezifische Funktion**, sondern ein allgemeiner Bestandteil des AGATE DSL und kann grundsätzlich bei unterschiedlichen Engine-Typen verwendet werden.

Beispiele:

```
1 - type: WAIT
2   condition: "{B[enabled]} == 'true'"
3   value: "1000"
```

```
1 - type: REST
2   condition: "{B[status]} != 'FAILED'"
3   op: EXEC
4   command: demo.request
```

```
1 - type: CALL
2   condition: "{R[userId]} != NULL"
3   command: reusable.demo
```

Auch die LOOP-Engine verwendet dieselbe Condition-Syntax.

---

## 1. Condition Handling – Kompakte System-Referenz für KI-Modelle

```
1 AGATE CONDITION HANDLING
2
3 PURPOSE:
4 Controls conditional execution of AGATE DSL steps.
5
6 FIELD:
7 condition: "...
8
9 GENERAL STEP SEMANTICS:
10
11 condition == true
12 -> execute step
13
14 condition == false
15 -> skip step
16
17 SKIPPED is not FAILED.
18
19 SUPPORTED BINARY OPERATORS:
20
21 Equality:
22 ==
23 =
24 EQUALS
25
26 Inequality:
27 !=
28 <>
29 NOT_EQUALS
30
31 String:
32 CONTAINS
33
34 SUPPORTED LOGICAL OPERATORS:
35
36 AND
37 OR
38 NOT
39
```

```
40 ALIASES:
41
42 && -> AND
43 || -> OR
44
45 SUPPORTED GROUPING:
46
47 (...)
48
49 SUPPORTED UNARY OPERATORS:
50
51 IS_NULL
52 IS_NOT_NULL
53 IS_EMPTY
54 IS_NOT_EMPTY
55
56 RECOMMENDED VALUE SOURCES:
57
58 {B[name]}
59 {R[name]}
60
61 IMPORTANT:
62
63 Binary comparisons use STRING semantics.
64
65 NOT SUPPORTED:
66
67 <
68 >
69 <=
70 >=
71
72 numeric ordering
73 arithmetic expressions
74 standalone truthy expressions
75
76 SAFE EXAMPLES:
77
78 condition: "{B[count]} == 0"
79
80 condition: "{B[status]} == 'READY'"
81
82 condition: "{B[status]} != 'FAILED'"
83
84 condition: "{B[text]} CONTAINS 'SUCCESS'"
85
86 condition: "{R[value]} != NULL"
87
88 condition: "{B[a]} == 'X' AND {B[b]} == 'Y'"
89
90 condition: "{B[a]} == 'X' OR {B[a]} == 'Y'"
91
92 condition: "NOT {B[status]} == 'FAILED'"
93
94 condition: "({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'"
95
96 AI GENERATION RULE:
97
98 Prefer small and explicit conditions.
99
100 Do not generate:
101 <
102 >
103 <=
104 >=
105 arithmetic
```

## 2. Grundprinzip

Die Condition wird vor der Ausführung eines normalen Steps ausgewertet.

Beispiel:

```
1 - type: WAIT
2   condition: "{B[enabled]}" == 'true'
3   value: "2000"
```

Wenn:

```
1 enabled = true
```

ergibt die Condition:

```
1 true
```

und der WAIT-Step wird ausgeführt.

Wenn:

```
1 enabled = false
```

wird der Step übersprungen.

Typischer Log:

```
1 >>> SKIPPED (Condition false: false == 'true')
```

Ein übersprungener Step ist kein Testfehler.

## 3. Placeholder Resolution

Vor der eigentlichen Condition-Auswertung werden AGATE-Platzhalter aufgelöst.

Beispiel:

```
1 condition: "{B[status]}" == 'READY'
```

Wenn:

```
1 B[status] = READY
```

wird der Ausdruck logisch zu:

```
1 READY == READY
```

und ergibt:

```
1 true
```

Typische Quellen sind:

```
1 {B[...] } Buffer / Runtime Value
2 {R[...] } Reusable Parameter
```

---

## 4. Equality

Unterstützte Operatoren:

```
1 ==
2 =
3 EQUALS
```

Diese Schreibweisen sind äquivalent.

Beispiele:

```
1 condition: "{B[status]} == 'READY'"
```

```
1 condition: "{B[status]} = 'READY'"
```

```
1 condition: "{B[status]} EQUALS 'READY'"
```

---

## 5. Inequality

Unterstützte Operatoren:

```
1 !=
2 <>
3 NOT_EQUALS
```

Beispiele:

```
1 condition: "{B[status]} != 'FAILED'"
```

```
1 condition: "{B[status]} <> 'FAILED'"
```

```
1 condition: "{B[status]} NOT_EQUALS 'FAILED'"
```

---

## 6. String-Vergleiche

Strings können mit einfachen oder doppelten Anführungszeichen verglichen werden.

Empfohlen:

```
1 condition: "{B[status]} == 'READY'"
```

Weitere Beispiele:

```
1 condition: "{B[result]} == 'SUCCESS'"
```

```
1 condition: "{R[action]} != 'DELETE'"
```

```
1 | condition: "{B[response_code]} == '200'"
```

---

## 7. Numerisch aussehende Werte

Folgendes ist gültig:

```
1 | condition: "{B[count]} == 0"
```

Aber dies ist **kein numerischer Vergleich**.

Der Evaluator behandelt beide Operanden als Strings.

Sinngemäß:

```
1 | "0" == "0"
```

Das bedeutet:

```
1 | condition: "{B[count]} == 0"
```

funktioniert.

Aber:

```
1 | condition: "{B[count]} > 0"
```

funktioniert nicht.

---

## 8. CONTAINS

**CONTAINS** prüft, ob der rechte String im linken String enthalten ist.

Beispiele:

```
1 | condition: "{B[message]} CONTAINS 'SUCCESS'"
```

```
1 | condition: "{B[response_body]} CONTAINS 'READY'"
```

```
1 | condition: "{B[log_output]} CONTAINS 'completed'"
```

```
1 | condition: "{B[file_content]} CONTAINS 'expected-value'"
```

**CONTAINS** ist case-sensitive.

```
1 | "SUCCESS" CONTAINS "SUCCESS"
2 | -> true
3 |
4 | "success" CONTAINS "SUCCESS"
5 | -> false
```

---

## 9. AND

Mehrere Bedingungen können mit **AND** verbunden werden.

```
1 | condition: "{B[status]} == 'READY' AND {B[result]} == 'OK'"
```

Weitere Beispiele:

```
1 | condition: "{R[user]} != NULL AND {R[action]} != NULL"
```

```
1 | condition: "{B[count]} == 0 AND {B[error]} != 'true'"
```

```
1 | condition: "{B[state]} == 'PROCESSING' AND {B[retry]} == 'true'"
```

```
1 | condition: "{R[enabled]} == 'true' AND {B[status]} != 'DONE'"
```

```
1 | condition: "{B[response]} != NULL AND {B[response]} CONTAINS 'RUNNING'"
```

Alle Teilbedingungen müssen **true** sein.

---

## 10. OR

**OR** ist ebenfalls unterstützt.

```
1 | condition: "{B[status]} == 'READY' OR {B[status]} == 'DONE'"
```

Weitere Beispiele:

```
1 | condition: "{B[state]} == 'FAILED' OR {B[state]} == 'ERROR'"
```

```
1 | condition: "{R[type]} == 'A' OR {R[type]} == 'B'"
```

```
1 | condition: "{B[result]} == 'PENDING' OR {B[result]} == 'PROCESSING'"
```

```
1 | condition: "{B[count]} == 0 OR {B[status]} == 'WAITING'"
```

Mindestens eine Teilbedingung muss **true** sein.

---

## 11. NOT

**NOT** negiert eine Condition.

Beispiele:

```
1 | condition: "NOT {B[status]} == 'FAILED'"
```

```
1 | condition: "NOT {B[result]} == 'ERROR'"
```

Mit Klammern:

```
1 | condition: "NOT ({B[status]} == 'FAILED' OR {B[status]} == 'ERROR')"
```

---

## 12. Klammern

Runde Klammern können zur Gruppierung verwendet werden.

```
1 | condition: "({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'"
```

Weitere Beispiele:

```
1 | condition: "({B[status]} == 'READY' OR {B[status]} == 'DONE') AND  
  {B[error]} != 'true'"
```

```
1 | condition: "{B[enabled]} == 'true' AND ({B[state]} == 'A' OR {B[state]}  
  == 'B')"
```

```
1 | condition: "NOT ({B[state]} == 'FAILED' OR {B[state]} == 'ERROR') AND  
  {B[ready]} == 'true'"
```

---

## 13. Operator-Priorität

Die effektive Priorität der aktuellen Auswertung ist:

```
1 | (...)
2 | NOT
3 | AND
4 | OR
```

Beispiel:

```
1 | A OR B AND C
```

wird interpretiert als:

```
1 | A OR (B AND C)
```

Für komplexere Expressions sollten trotzdem explizite Klammern verwendet werden.

---

## 14. && und ||

Die Kurzformen:

```
1 | &&
2 | ||
```

werden vor der Auswertung normalisiert.

Beispiel:

```
1 | condition: "{B[a]} == '1' && {B[b]} == '2'"
```

entspricht:

```
1 | condition: "{B[a]} == '1' AND {B[b]} == '2'"
```

Und:

```
1 | condition: "{B[a]} == '1' || {B[b]} == '2'"
```

entspricht:

```
1 | condition: "{B[a]} == '1' OR {B[b]} == '2'"
```

Für öffentliche Testfälle wird **AND** / **OR** empfohlen, da dies besser lesbar ist.

---

## 15. NULL

Für NULL-Vergleiche ist folgende Syntax etabliert:

```
1 | condition: "{B[value]} == NULL"
```

```
1 | condition: "{B[value]} != NULL"
```

Weitere Beispiele:

```
1 | condition: "{R[userId]} != NULL"
```

```
1 | condition: "{R[a]} != NULL AND {R[b]} != NULL"
```

```
1 | condition: "{B[result]} == NULL OR {B[result]} == 'PENDING'"
```

Empfohlen ist:

```
1 | NULL
```

ohne String-Quotes zu verwenden.

---

## 16. IS\_EMPTY und IS\_NOT\_EMPTY

Der Evaluator unterstützt:

```
1 | IS_EMPTY  
2 | IS_NOT_EMPTY
```

Beispiele:

```
1 | condition: "{B[result]} IS_EMPTY"
```

```
1 | condition: "{B[result]} IS_NOT_EMPTY"
```

```
1 | condition: "{B[response]} IS_NOT_EMPTY AND {B[status]} == 'READY'"
```

---

## 17. IS\_NULL und IS\_NOT\_NULL

Die Syntax ist im Evaluator vorhanden:

```
1 | IS_NULL  
2 | IS_NOT_NULL
```

Beispiele:

```
1 | condition: "{B[value]} IS_NULL"
```

```
1 | condition: "{B[value]} IS_NOT_NULL"
```

Für die derzeitige DSL-Generierung wird jedoch bevorzugt:

```
1 | condition: "{B[value]} == NULL"
```

bzw.

```
1 | condition: "{B[value]} != NULL"
```

verwendet.

---

## 18. Case Sensitivity

Operatoren werden case-insensitive erkannt.

Beispielsweise:

```
1 | AND
2 | and
3 | And
```

sind syntaktisch äquivalent.

Dasselbe gilt für:

```
1 | EQUALS
2 | equals
3 | Equals
```

Die verglichenen Werte selbst sind jedoch case-sensitive.

```
1 | READY == READY
2 | -> true
3 |
4 | ready == READY
5 | -> false
```

---

## 19. Nicht unterstützte Operatoren

Nicht unterstützt:

```
1 | <
2 | >
3 | <=
4 | >=
```

Daher nicht verwenden:

```
1 | condition: "{B[count]} > 5"
```

```
1 | condition: "{B[count]} < 10"
```

```
1 | condition: "{B[count]} >= 3"
```

```
1 | condition: "{B[count]} <= 20"
```

---

## 20. Keine Arithmetik

Nicht unterstützt:

```
1 | condition: "{B[counter]} + 1 == 5"
```

```
1 | condition: "{B[value]} * 2 == 10"
```

```
1 | condition: "{B[value]} / 2 == 4"
```

Der ConditionEvaluator ist keine mathematische Expression Engine.

---

## 21. Keine Standalone-Boolean-Expressions

Nicht verwenden:

```
1 | condition: "true"
```

Nicht verwenden:

```
1 | condition: "{B[enabled]}"
```

Stattdessen explizit vergleichen:

```
1 | condition: "{B[enabled]} == 'true'"
```

---

## 22. Viele praktische Beispiele

Status ist READY

```
1 | condition: "{B[status]} == 'READY'"
```

Status ist nicht READY

```
1 | condition: "{B[status]} != 'READY'"
```

Status ist READY oder DONE

```
1 | condition: "{B[status]} == 'READY' OR {B[status]} == 'DONE'"
```

Status ist weder FAILED noch ERROR

```
1 | condition: "NOT ({B[status]} == 'FAILED' OR {B[status]} == 'ERROR')"
```

Zwei Werte müssen vorhanden sein

```
1 | condition: "{R[userId]} != NULL AND {R[action]} != NULL"
```

### Text enthält SUCCESS

```
1 | condition: "{B[text]} CONTAINS 'SUCCESS'"
```

### Text enthält keinen ERROR

```
1 | condition: "NOT {B[text]} CONTAINS 'ERROR'"
```

### Ergebnis ist leer

```
1 | condition: "{B[result]} IS_EMPTY"
```

### Ergebnis ist nicht leer

```
1 | condition: "{B[result]} IS_NOT_EMPTY"
```

### Count ist 0

```
1 | condition: "{B[count]} == 0"
```

### Count ist nicht 0

```
1 | condition: "{B[count]} != 0"
```

### Feature ist aktiv und Status READY

```
1 | condition: "{B[enabled]} == 'true' AND {B[status]} == 'READY'"
```

### Einer von zwei Prozessen läuft

```
1 | condition: "{B[processA]} == 'RUNNING' OR {B[processB]} == 'RUNNING'"
```

### Status ist PENDING oder PROCESSING

```
1 | condition: "{B[status]} == 'PENDING' OR {B[status]} == 'PROCESSING'"
```

### Antwort vorhanden und enthält READY

```
1 | condition: "{B[response]} != NULL AND {B[response]} CONTAINS 'READY'"
```

### Komplexere Kombination

```
1 | condition: "({B[state]} == 'READY' OR {B[state]} == 'DONE') AND  
{B[error]} != 'true'"
```

---

## 23. Verhalten bei normalen Steps

Für normale Steps:

```
1 | condition true  
2 | -> Step ausführen  
3 |  
4 | condition false  
5 | -> Step SKIPPED
```

Beispiel:

```
1 - type: WAIT
2   condition: "{B[enabled]} == 'true'"
3   value: "1000"
```

oder:

```
1 - type: CALL
2   condition: "{R[userId]} != NULL"
3   command: reusable.demo
```

---

## 24. Verwendung durch Control-Flow-Engines

Control-Flow-Komponenten wie LOOP verwenden dieselbe zentrale Condition-Syntax.

Die konkrete Bedeutung von `true` und `false` kann dabei Engine-spezifisch sein.

Bei normalen Steps:

```
1 true -> execute
2 false -> SKIPPED
```

Bei LOOP:

```
1 true -> another iteration
2 false -> LOOP end
```

Die LOOP-spezifische Semantik ist im separaten Dokument **LOOP Engine** beschrieben.

---

## 25. Bekannte Parser-Einschränkung

Die aktuelle Logik berücksichtigt beim Aufteilen von `AND` und `OR` Klammerung, aber keine String-Quotes.

Daher sollten Stringwerte möglichst nicht selbst:

```
1 AND
```

oder:

```
1 OR
```

enthalten.

Nicht empfohlen:

```
1 condition: "{B[text]} == 'A OR B'"
```

---

## 26. Best Practices

Conditions möglichst klein halten.

Bevorzugt:

```
1 | condition: "{B[status]} == 'READY'"
```

oder:

```
1 | condition: "{B[status]} != 'FAILED' AND {B[result]} == 'OK'"
```

Für technisch erzeugte Werte bevorzugt:

```
1 | ENGINE
2 |   ↓
3 | BUFFER
4 |   ↓
5 | CONDITION
```

Dadurch bleibt Condition Handling unabhängig von der konkreten technischen Engine.

---

## 27. Troubleshooting

Wenn eine Condition unerwartet `false` ergibt:

1. Prüfen, ob alle Placeholder aufgelöst wurden.
  2. Prüfen, welcher tatsächliche Buffer-Wert vorhanden ist.
  3. Groß-/Kleinschreibung prüfen.
  4. Quotes prüfen.
  5. NULL prüfen.
  6. Sicherstellen, dass nur unterstützte Operatoren verwendet werden.
  7. Prüfen, ob der benötigte Wert bereits existiert.
- 

## 28. Kurzreferenz für KI / RAG

```
1 | AGATE CONDITION HANDLING
2 |
3 | GLOBAL DSL FEATURE:
4 | Condition is not tied to one engine.
5 |
6 | FIELD:
7 | condition
8 |
9 | NORMAL STEP:
10 | true  -> execute
11 | false -> SKIPPED
12 |
13 | SUPPORTED:
14 |
15 | ==
16 | =
17 | EQUALS
18 |
19 | !=
20 | <>
```

```
21 NOT_EQUALS
22
23 CONTAINS
24
25 AND
26 OR
27 NOT
28
29 &&
30 ||
31
32 (...)
33
34 IS_NULL
35 IS_NOT_NULL
36 IS_EMPTY
37 IS_NOT_EMPTY
38
39 RECOMMENDED NULL:
40 == NULL
41 != NULL
42
43 VALUES:
44 binary comparisons are string based
45
46 CASE:
47 operator names case-insensitive
48 values case-sensitive
49
50 NOT SUPPORTED:
51 <
52 >
53 <=
54 >=
55 arithmetic
56 numeric ordering
57 standalone truthy expressions
58
59 SAFE:
60 {B[status]} == 'READY'
61 {B[status]} != 'FAILED'
62 {B[count]} == 0
63 {B[text]} CONTAINS 'SUCCESS'
64 {R[value]} != NULL
65 {B[a]} == 'X' AND {B[b]} == 'Y'
66 {B[a]} == 'X' OR {B[a]} == 'Y'
67 NOT {B[state]} == 'FAILED'
68 ({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'
69
70 PREFERRED DESIGN:
71 ENGINE
72 -> BUFFER
73 -> CONDITION
74
75 AI RULE:
76 Never invent unsupported numerical comparison operators.
```